PHASE-2 · PROJECT IMPLEMENTATION · DL MINI PROJECT

IndicDoc AI

Where do layout detectors fail on Indian documents? A study of script, domain and scan quality

Prashant Jha — Roll No. 41

Aditya Soni — Roll No. 42

Riddhi Patil — Roll No. 39

Mariam Badure — Roll No. 40

Course: Deep Learning Mini Project (Expt. 11, SDG-based)

Department: Artificial Intelligence and Machine Learning, St. Francis Institute of Technology (SFIT)

Guide: Dr. Joanne Gomes

Academic year: 2026–27 · Semester V (TE AIML)



OVERVIEW

Content

01 Literature Survey Summary Table

02 Research Gaps

03 Problem Statement and Objectives

04 Proposed System, Methodology and Novelty

05 System Block Diagram

06 Dataset Description and Pre-processing

07 Deep Learning Model Architecture

08 Tools, Technologies and Environment

09 Implementation Details

10 Experimental Setup and Training

11 Results and Performance Evaluation

12 Comparison with Baseline / Existing Methods

13 Sample Outputs and Demonstration

14 Challenges and Limitations

15 Conclusion and Future Scope

16 References

PHASE-1 RECAP

Literature Survey Summary Table

Ref	Author	Problem	Dataset	Method	Limitation
[1]	**Nath et al., 2025**	Layout parsing of Indian-language pages	IndicDLP: 119,806 pages, 12 languages, 12 domains, 42 labels	Benchmark of YOLOv10x, DocLayout-YOLO, RoDLA, DINO, DiT, Florence-2	Vision-only; 20–25 mAP drop on unseen scripts; no robustness test
[2]	**Kasuba et al., 2025**	Page-level handwritten Indic text recognition	CHIPS, ~25K pages, 10 languages	DBNet word detection + 6 recognisers (CRNN, PARSeq, ...)	Synthetic pages; user must name the language
[3]	**Mondal et al., 2026**	Multilingual table detection	UniTabBank: 55K pages, 82K tables, 28 languages	Semi-automatic labelling; YOLOv11 detector	Borderless tables and degraded scans fail
[4]	**Pal et al., 2024**	Table question answering, low-resource Indic	BanglaTabQA 2M, HindiTabQA 643K	SQL-generated QA; fine-tuned mBART / M2M / Llama	Wikipedia tables only; small test sets
[5]	**Xu et al., 2021**	Multilingual document understanding	30M docs; XFUND, 7 languages	LayoutXLM: text + layout + image transformer	No Indian language; needs good OCR
[6]	**Kasuba et al., 2025**	Grounding answers on the page	MGVG: 70 circulars, 509 QA pairs	OCR + Llama-3.1-8B + region matching	OCR-dependent; single page; small test set
[7]	**Bhattacharyya et al., 2025**	Hindi OCR with vision-language models	1.1M printed Hindi words + pages	LoRA-tuned GOT-OCR 2.0, progressive learning	Hindi only; mixed-language lines
[8]	**Dumada & Shah, 2026**	Devanagari table extraction	500+ Devanagari documents	Structure + Tesseract content, weighted fusion	Self-reported results; no ablation

WHAT IS MISSING

Research Gaps

⚠ Only large models were benchmarked, on clean pages	→	✓ **No study of how accuracy holds up on blurred, noisy, compressed or faded scans** — RQ4
⚠ Results are one overall number	→	✓ **Is accuracy even across 12 languages and 12 domains? Which labels fail, and why?** — RQ2, RQ3
⚠ OCR tools expect the user to name the language	→	✓ **Automatic script choice per page, using the detected regions** — RQ5

Our position: we do not claim a new detector. Strong detectors already exist; our contribution is a careful **study** of where they fail on Indian documents, plus a working tool built on the best one.

WHAT WE SOLVE

Problem Statement and Objectives

Problem: Indian offices, schools and archives hold millions of scanned pages in 12+ scripts. To search or reuse them, a system must first find every region on the page (paragraph, heading, table, figure ...) and then read the text. Most tools are built and tested on English.

MAIN OBJECTIVE

Measure, in simple numbers, where deep-learning layout detectors work and where they fail on Indian-language pages, and deliver a usable tool.

SPECIFIC OBJECTIVES

- Train two detectors (Faster R-CNN, YOLOv8s) and compare them with the published IndicDLP model on the same test pages.
- Define two simple measures: Consistency Index and Robustness Index.
- Add OCR with automatic script detection for 12 languages.
- Ship a web app + REST API in Docker.

EXPECTED OUTPUT

A labelled box for every region, the page text in reading order (Markdown / JSON), and the study results.

RESEARCH QUESTIONS

RQ1 How far behind is a detector trained on a small budget, compared with one trained on the full dataset?

RQ2 Is accuracy even across the 12 languages and 12 document domains?

RQ3 Which region types are hardest, and is that because they are rare?

RQ4 How much accuracy survives blur, noise, compression, low resolution and fading?

RQ5 Can detected regions choose the right OCR language automatically?

OUR APPROACH

Proposed System, Methodology and Novelty

1. **Page in**
   image or PDF, any of 12 languages
2. **Detect regions**
   DocLayout-YOLO, 42 labels
3. **Clean up**
   confidence threshold, remove duplicates
4. **Pick script**
   probe each script on one region
5. **Read text**
   Tesseract, one pass per page
6. **Order**
   words → regions → reading order
7. **Output**
   boxes + Markdown / JSON

How deep learning is used

Convolutional object detectors look at the page image and output a box, a label and a confidence for every region. We fine-tune COCO-pretrained detectors (transfer learning) and use the published IndicDLP model.

What is new (the study)

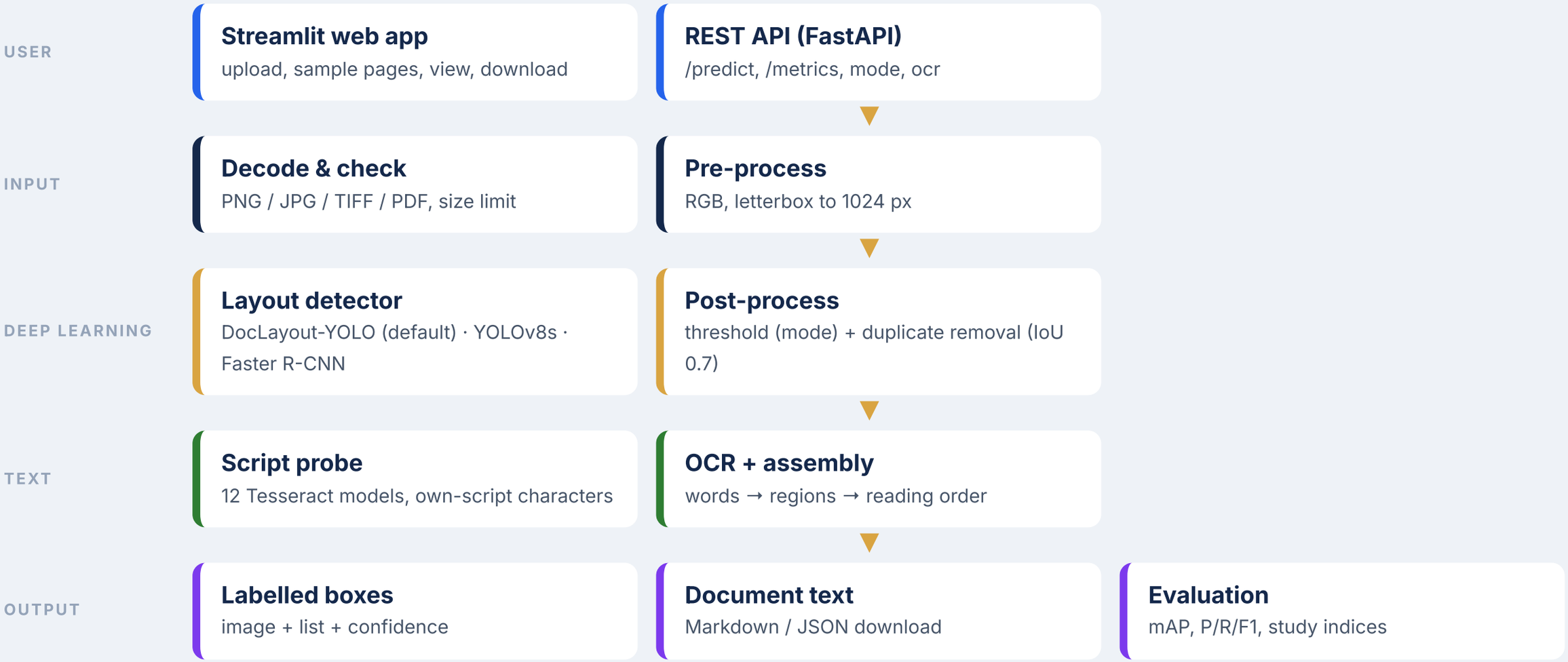
Consistency Index CI = worst group mAP / best group mAP
Robustness Index RI = damaged mAP / clean mAP
First scan-damage test on IndicDLP: blur, noise, JPEG, half resolution, faded print

What is new (the tool)

Two operating modes picked on validation pages ("find every part", "balanced"); script detection by probing detected regions (106/120 vs 70/120 for Tesseract OSD); one app for 12 languages.

ARCHITECTURE

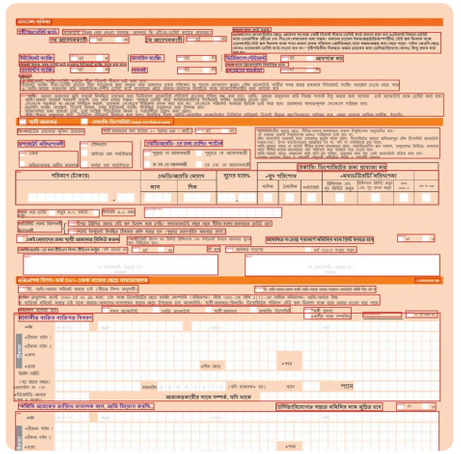
System Block Diagram



DATA

Dataset Description and Pre-processing

Dataset	**IndicDLP** (Nath et al., ICDAR 2025) — Hugging Face `ai4bharat/indicdlp`, also on AIKosh; MIT licence
Full size	119,806 pages · 12 languages · 12 domains · 42 region labels
Our subset	**3,317 pages**, 49,995 boxes, balanced by language (~200 train pages each), taken from the official train / val / test splits
Split	train **2,293** · validation **342** · test **682** pages
Pre-processing	Parquet → PNG + COCO JSON; boxes clipped to the page, <1 px boxes dropped (0.5%); COCO → YOLO labels; resize / letterbox (640 px ours, 1024 px authors)
Augmentation	YOLOv8: mosaic, HSV, scale/translate. Faster R-CNN: colour jitter, blur. **No mirror flips** (a mirrored page is not real)
Challenges	**Heavy class imbalance:** "paragraph" = 32% of all boxes; 10 of 42 labels have <100 training boxes. Hindi has only 22 test pages.
Deliverable	`dataset/`: pages.csv, boxes.csv, class_counts.csv, language_split.csv + one sample per language



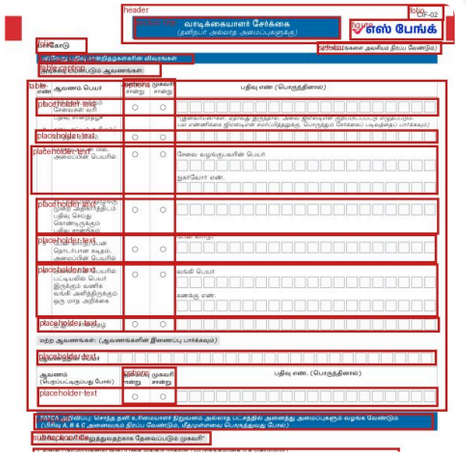
bengali · forms



hindi · newspapers



punjabi · brochures



tamil · forms

MODELS

Deep Learning Model Architecture

DocLayout-YOLO · 20.0 M params · default

Input 1024×1024×3 page → **Backbone** CSP conv blocks + GL-CRM (global-to-local receptive fields) → **Neck** PAN feature pyramid, 3 scales → **Head** YOLOv10-style, NMS-free → **Output** N × (box, 42-class score)

Pre-trained on DocSynth-300K, fine-tuned on the full IndicDLP train set by its authors (Zenodo 15881917, CC-BY 4.0). Used as released; we did not change it.

YOLOv8s · 11.2 M params · trained by us

Input 640×640×3 → **Backbone** CSPDarknet with C2f blocks + SPPF → **Neck** PAN-FPN → **Head** decoupled, anchor-free → **Output** boxes + 42 classes

Transfer learning: COCO-pretrained weights; the 80-class head is replaced with a 42-class head. Loss: CIoU + DFL (boxes) + BCE (classes).

Faster R-CNN · 41.5 M params · baseline

Input longest side 640–1024 → **Backbone** ResNet-50, frozen batch-norm → **Neck** FPN, 5 levels → **RPN + RoIAlign** region proposals → **Box head** 43 outputs (42 + background)

Two-stage detector. COCO-pretrained; the predictor is replaced (FastRCNNPredictor, 43 classes). Loss: RPN + classification CE + smooth-L1 box.

STACK

Tools, Technologies and Environment

Component	Details
Programming language	Python 3.11 (ML, app, API); JavaScript only for an unused React prototype
DL framework	PyTorch 2 with torchvision; Ultralytics 8 (YOLOv8); doclayout-yolo 0.0.4
Development environment	VS Code / terminal, Git + GitHub (private repo), pytest (141 tests)
Libraries	pycocotools (COCO mAP), OpenCV, Pillow, NumPy, pandas, Matplotlib, Plotly, PyMuPDF, Tesseract 5 (tessdata_best, 12 scripts)
Hardware	Apple M1 Pro MacBook, 16-core GPU via PyTorch MPS (training + evaluation); CPU in Docker
Dataset tools	Hugging Face Hub + fsspec (partial Parquet reads), custom COCO / YOLO converters
Interface / deployment	Streamlit web app + FastAPI REST API, Docker + docker-compose (works offline)

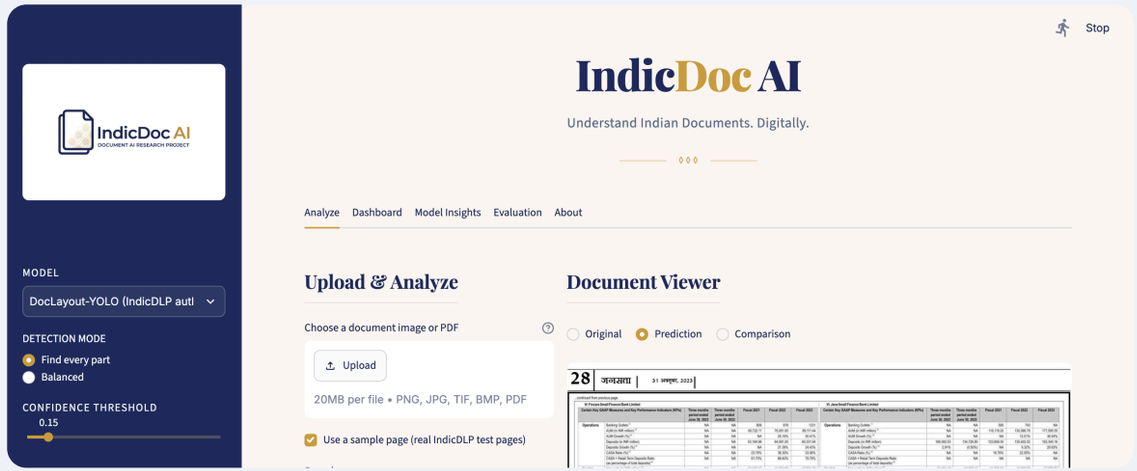
WHAT WE BUILT

Implementation Details

- **Data loading:** Parquet row-groups streamed from Hugging Face, language-balanced sampling, COCO JSON + YOLO labels.
- **Model construction:** COCO-pretrained YOLOv8s and Faster R-CNN, new 42-class heads; the authors' DocLayout-YOLO loaded as released.
- **Loss / optimiser:** Faster R-CNN — SGD lr 0.005, momentum 0.9, weight decay 5e-4, cosine schedule. YOLOv8s — Ultralytics automatic optimiser, CIoU + DFL + BCE loss.
- **Batch / epochs:** Faster R-CNN batch 4, 2 epochs (time budget); YOLOv8s batch 8, 12 epochs (1-hour budget). Seed 42.
- **Validation:** best checkpoint on 342 validation pages; operating thresholds chosen on validation, never on test.
- **Inference:** one page at a time, duplicate boxes removed, two modes: "find every part" (conf 0.15) and "balanced" (conf 0.35).
- **Integration:** same code in the Streamlit app, the FastAPI service and the evaluation scripts; Docker image runs with no internet.

```python
# ml/inference/predict.py — one page in, labelled boxes out
model = load_model(ckpt, "doclayout", device)
out = predict_single(model, page, "doclayout", device, conf=0.15)
out = dedupe_boxes(out, iou_threshold=0.7)

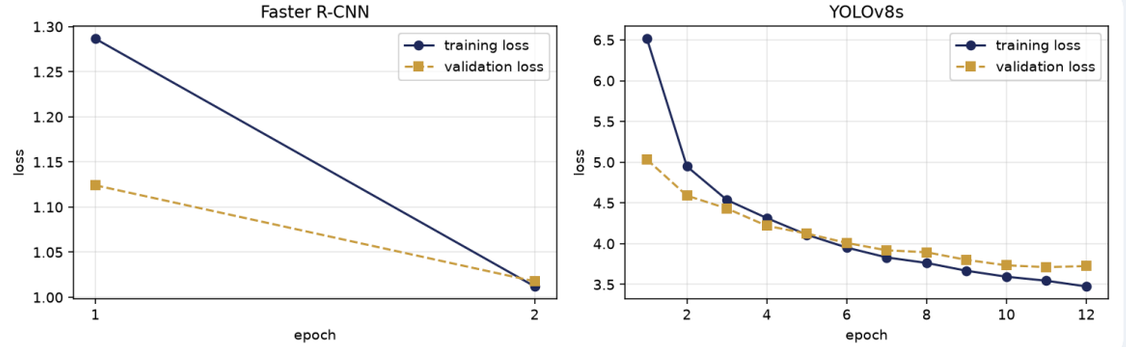
# ml/ocr — read the text inside the regions
doc = extract_document(page, out, languages="auto")
print(doc["markdown"])   # text in reading order
```



EXPERIMENTS

Experimental Setup and Training

Data sizes	train 2,293 · val 342 · test 682 pages (test = 10,513 boxes)
Faster R-CNN	ResNet-50 FPN, 640–1024 px, batch 4, SGD 0.005, **2 epochs, 76 min**
YOLOv8s	640 px, batch 8, mosaic on, no flips, **12 epochs, 60 min**
DocLayout-YOLO	1024 px, released checkpoint, evaluated only
Hardware	Apple M1 Pro GPU (MPS), one page at a time for timing
Metrics	COCO mAP@50 and mAP@50:95 (pycocotools, 300 boxes/page), precision, recall, F1
Study runs	**E1** 3 models on clean pages: per language, domain, label **E2** DocLayout-YOLO and YOLOv8s × 6 page conditions (clean + 5 damage types) × 682 pages **E3** OCR script detection on 120 pages (10 per language)



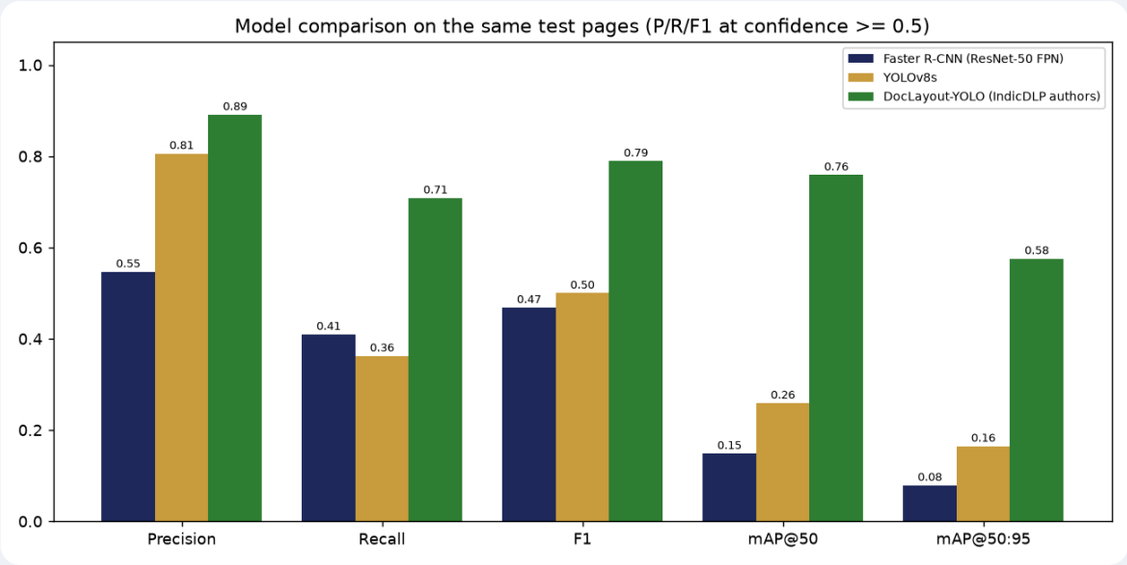
Training and validation loss of our two models (measured).

RESULTS

Results and Performance Evaluation

Model (682 test pages)	mAP@50	mAP@50:95	Precision*	Recall*	F1*	Recall, find-every-part	s / page	MB
DocLayout-YOLO (authors)	0.759	0.575	0.848	0.775	0.810	0.851	0.150	162
YOLOv8s (ours)	0.259	0.165	0.603	0.508	0.551	0.508	0.024	22
Faster R-CNN (ours)	0.148	0.079	0.510	0.435	0.469	0.435	0.202	167

* at the "balanced" threshold chosen on validation pages. Timing on Apple M1 Pro GPU.



0.759

mAP@50, best model (paper reports ≈0.735 on its full test set)

85%

of all regions found in "find every part" mode, at 74% precision

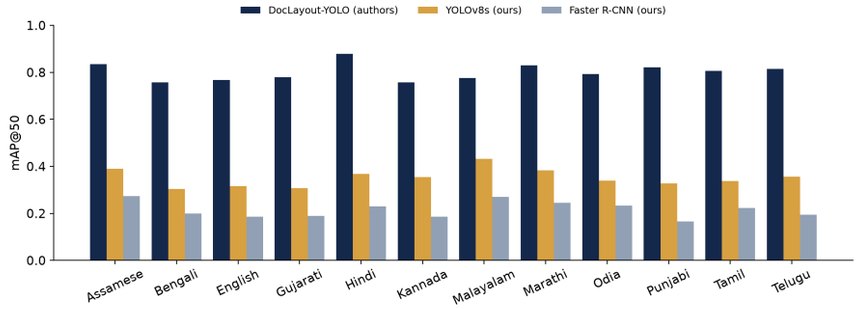
106/120

pages: OCR script picked correctly (Tesseract's own detector: 70/120)

STUDY FINDINGS

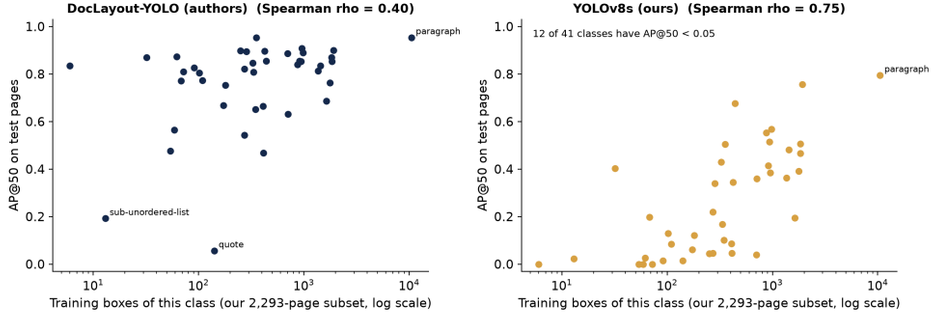
Results: Where Do Detectors Fail?

RQ2 · Languages and domains



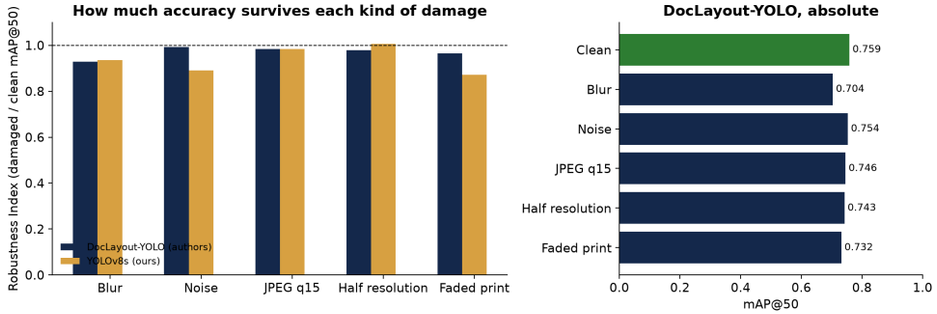
Language CI: authors **0.86**, YOLOv8s 0.70, Faster R-CNN 0.61. Domain CI (authors): **0.55** — the **domain** matters more than the script.

RQ3 · Rare labels



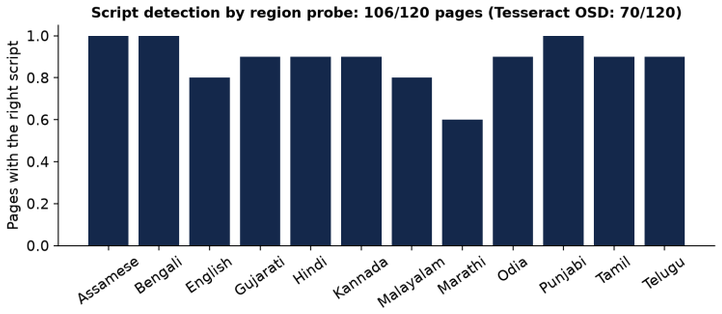
Rank correlation of class frequency with AP: **0.75** for our small model, only 0.40 for the full-data model. More data fixes rarity.

RQ4 · Damaged scans



Mean Robustness Index: authors **0.97**, YOLOv8s 0.94. Blur hurts most (RI 0.93).

RQ5 · Choosing the OCR script



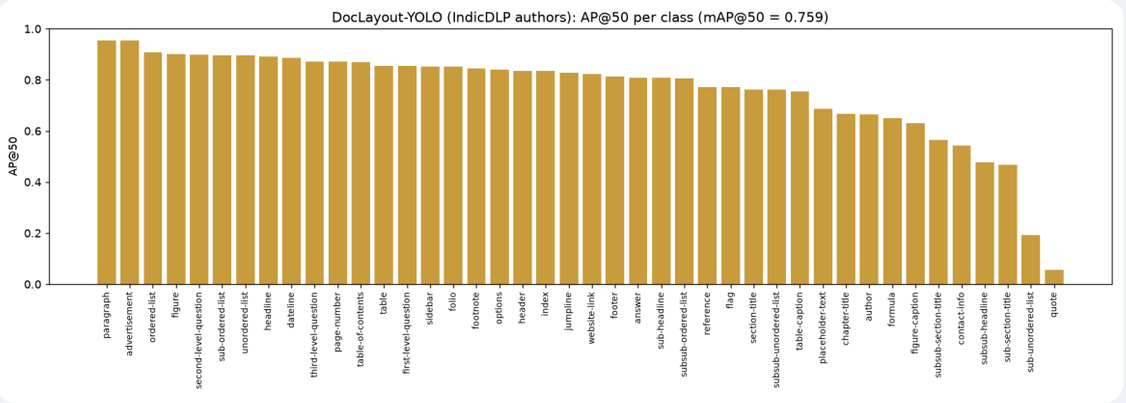
Region probe picks the right script on **106/120** pages vs 70/120 for Tesseract's own detector.

COMPARISON

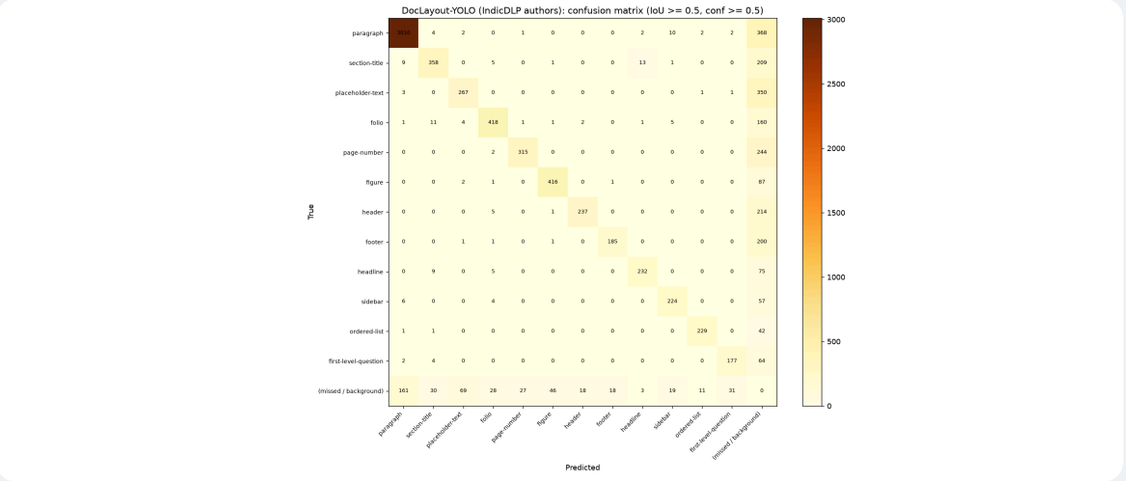
Comparison with Baseline / Existing Methods

Method	Training data	mAP@50	mAP@50:95
Faster R-CNN (ours, baseline)	2,293 pages, 76 min	0.148	0.079
YOLOv8s (ours)	2,293 pages, 60 min	0.259	0.165
DocLayout-YOLO (IndicDLP authors)	~96K pages (80% of 119,806)	**0.759**	**0.575**
Published: DocLayout-YOLO [1]	*full IndicDLP test set*	*≈0.735*	*0.545*
Published: YOLOv10x [1] (best in paper)	*full IndicDLP test set*	–	*0.550*

- **Metrics:** COCO mAP@50, mAP@50:95, precision/recall/F1, seconds per page — all on the same 682 test pages.
- **One-stage vs two-stage:** with the same data and similar time, YOLOv8s beats Faster R-CNN (0.259 vs 0.148) and is 9× faster.
- **Did we improve?** Our own models did not beat the published model — they saw ~42× less data. So the app uses the published model, and our contribution is the study and the OCR pipeline. This is stated honestly in the paper.

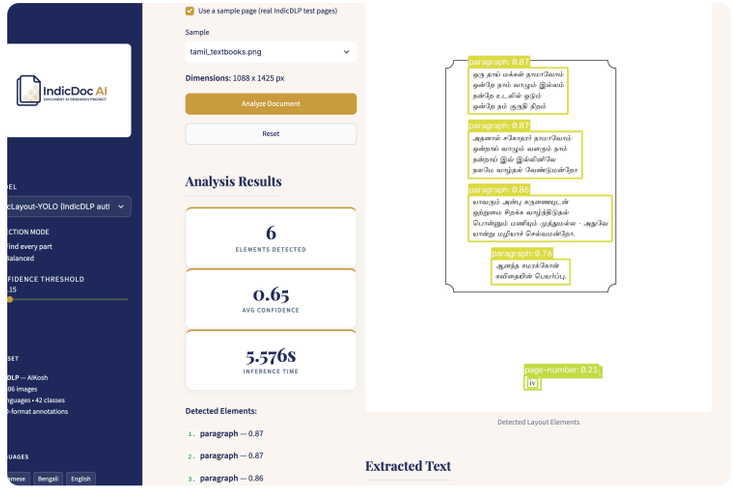


AP@50 per label, DocLayout-YOLO.

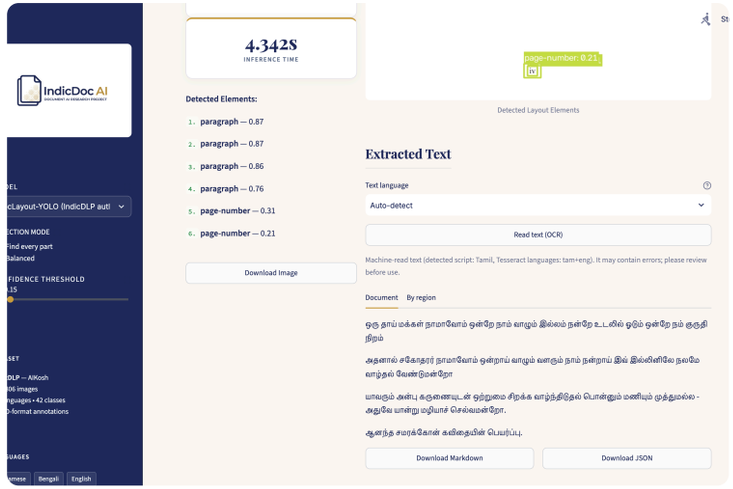


DEMO

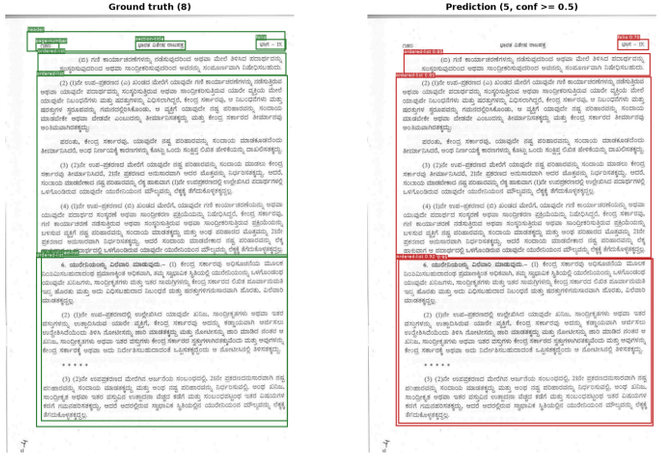
Sample Outputs and Demonstration



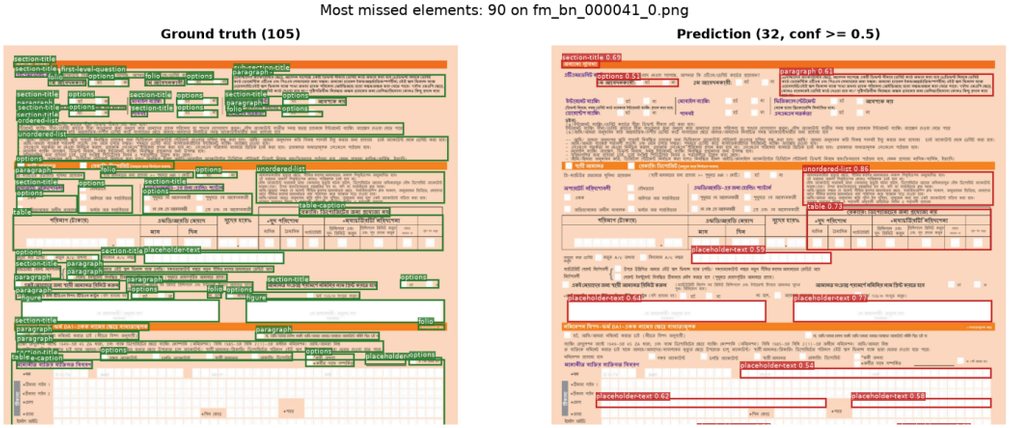
Running app: Tamil textbook page, 6 regions found



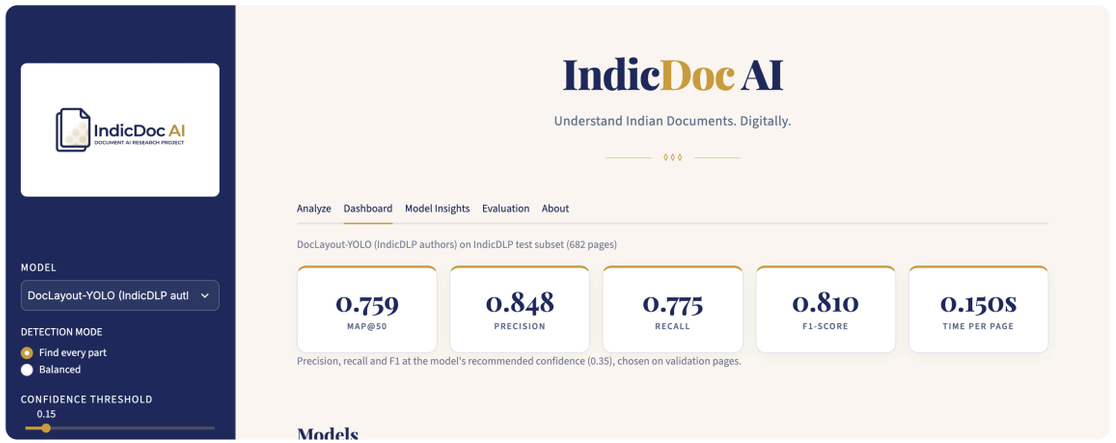
OCR: script auto-detected as Tamil, text in reading order, MD / JSON download



Correct: Kannada act — ground truth (green) vs prediction (red)



Incorrect: dense Bengali form — 90 of 105 regions missed (forms are the hardest domain)



Dashboard tab: measured metrics of all three models

LESSONS

Challenges and Limitations

Data size and compute

The full dataset is 91 GB; a laptop GPU can train on ~2.3K pages for an hour. Our own models stay far below the published one (0.26 vs 0.76 mAP@50).

Dense forms and rare labels

Forms, with many tiny fields, score lowest; labels like quote and sub-unordered-list score below 0.2 AP even for the best model.

OCR quality

Tesseract misreads small print and some scripts (e.g. Marathi script picked right on 6/10 pages); IndicDLP has no text ground truth, so character accuracy could not be measured.

Engineering bugs found and fixed

The original code had crashes, wrong label lists and made-up plots; 30 fixes are logged in docs/FINAL_AUDIT.md. Timings were spoiled twice by jobs sharing the GPU and had to be re-run.

WRAP-UP

Conclusion and Future Scope

CONCLUSION

- **Implemented:** a full pipeline — data preparation, two trained detectors, evaluation of three, OCR for 12 scripts, web app, REST API, Docker.
- **Main results:** best model 0.759 mAP@50 on 682 test pages, finds 85% of regions in "find every part" mode, 0.15 s/page; script picked right on 106/120 pages.
- **Study:** accuracy is fairly even across languages (CI 0.86) but not across domains (CI 0.55); data size, not the script, decides how well rare labels work; the best model keeps 97% of its accuracy on damaged scans (blur hurts most).
- **Objectives:** all four met; our own detectors did not reach the published accuracy, which we report openly.

FUTURE SCOPE

- Controlled unseen-script test (same data, with / without a script family); train longer on a cloud GPU with blur augmentation.
- Replace Tesseract with an Indic OCR model (e.g. a fine-tuned VLM) and build a small text ground-truth set.
- Table structure and question answering on detected tables [3, 4]; answer grounding [6].
- Real-world pilot on government circulars and school textbooks; mobile / edge model.

SDG 4 | SDG 9 | SDG 10 | SDG 16

SOURCES

References

1. O. Nath, S. Kukkala, M. Khapra, R. K. Sarvadevabhatla, "IndicDLP: A Foundational Dataset for Multi-Lingual and Multi-Domain Document Layout Parsing," ICDAR 2025; arXiv:2512.20236.
2. B. V. Kasuba et al., "PLATTER: A Page-Level Handwritten Text Recognition System for Indic Scripts," arXiv:2502.06172, 2025.
3. A. Mondal et al., "UniTabBank: A Large Scale Multi-Lingual, Multi-Layout, Multi-Type, Multi-Format Dataset for Table Detection," IEEE/CVF WACV 2026.
4. V. Pal, E. Kanoulas, A. Yates, M. de Rijke, "Table Question Answering for Low-resourced Indic Languages," arXiv:2410.03576, 2024.
5. Y. Xu et al., "LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding," arXiv:2104.08836, 2021.
6. B. V. Kasuba, P. Chaudhuri, G. Ramakrishnan, "DRISHTIKON: Visual Grounding at Multiple Granularities in Documents," arXiv:2506.21316, 2025.
7. S. Bhattacharyya et al., "Adapting Vision-Language Models for Hindi OCR," IIIT Hyderabad, 2025.
8. A. R. Dumada, S. G. Shah, "Incorporate Structure and Content for Devanagari Table Extraction," IJSAT 17(1), 2026.
9. Z. Zhao et al., "DocLayout-YOLO: Enhancing Document Layout Analysis through Diverse Synthetic Data and Global-to-Local Adaptive Perception," arXiv:2410.12628, 2024.
10. S. Ren, K. He, R. Girshick, J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," NeurIPS 2015.
11. G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," 2023, github.com/ultralytics/ultralytics.
12. T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," ECCV 2014 (metric and pretrained weights).
13. R. Smith, "An Overview of the Tesseract OCR Engine," ICDAR 2007; tessdata_best models.
14. D. Hendrycks, T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," ICLR 2019.
15. PyTorch, torchvision, pycocotools, Streamlit, FastAPI, Docker — official documentation.

OTHER SOURCES

Dataset: huggingface.co/datasets/ai4bharat/indicdlp (AIKosh)
Model weights: zenodo.org/records/15881917
Code: github.com/va-ada/IndicDoc-AI (private)
indicdlp.github.io

Thank You

Questions and discussion

Prashant Jha | Aditya Soni | Riddhi Patil | Mariam Badure

IndicDoc AI | Phase-2: Project Implementation